RV Institute of Technology and Management®

Department of Master of Computer Applications

LOW-LEVEL DESIGN (LLD)

AI Emergency Response Platform

Status note. This document describes the *proposed* design. Collections, routes and rules listed here are planned; none is claimed as implemented. Values such as file-size limits and token expiry are proposed defaults that may change during development.

1. HLD versus LLD

Aspect	High-Level Design (HLD)	Low-Level Design (LLD)
Question answered	Which main components exist and how do they connect?	How exactly does each component work inside?
Level of detail	Boxes and arrows: frontend, API, AI adapter, database	Fields, routes, validation rules, status transitions, error codes
Example from this project	“The backend sends the report to the AI Analysis Adapter and stores the result.”	“The adapter returns JSON; the category must be one of 7 allowed values, otherwise aiStatus is set to failed.”

2. Module-wise Design

Module and purpose	Validation and errors	Access
1. Authentication and user management Register, log in, manage accounts. Input: name, email, password, confirm password. Output: JWT and user profile.	Email format and unique; password at least 8 characters; confirm matches; password stored as hash. Errors: 400 invalid input, 409 email exists, 401 wrong email or password (same message for both).	Register and login: public. Profile: any logged-in user. Account management: Admin.
2. Incident reporting Create and view reports. Input: description, optional image, location fields, urgent flag. Output: reportId and saved incident.	Description required (trimmed, max 2000 characters); at least coordinates or a location description. Errors: 400, 401. The report is saved before AI runs.	User
3. Location selection Capture location by GPS, map click or typed text. Output: latitude, longitude, locationSource (gps, map, manual), locationDescription.	Latitude -90 to 90, longitude -180 to 180. If GPS permission is denied, show a message and keep the map and manual options available.	User (frontend component)
4. Image upload Attach one optional photo. Input: JPG, JPEG or PNG file. Output: stored path in imageUrl.	Type checked by extension and content type; size at most 5 MB (proposed); file renamed safely. Errors: 415 wrong type, 413 too large.	User
5. AI analysis Recommend category, priority, summary and reason. Input: description, optional image, location text. Output: aiCategory, aiPriority, aiSummary, aiReason, aiConfidence (only if the model gives a meaningful value).	Reply must be valid JSON with an allowed category and priority. Timeout or invalid reply sets aiStatus to failed; the report stays Submitted for manual review. AI never writes to the database or changes permissions.	Internal only (called by the backend; no public AI route)
6. Incident assignment Route the report to a responder group, or assign a person. Output: assignedCategory, assignedResponderId, status.	Responder must exist, be active and match the category. First accept wins (atomic update). Errors: 404 not found, 409 already accepted.	System (routing); Admin (manual assign or reassign)

Module and purpose	Validation and errors	Access
7. Responder dashboard Show incidents for the responder's group and those assigned to them, with description, image, map and AI analysis.	Role check; responder sees only their group or assigned incidents. Error: 403.	Responder
8. Status tracking and history Change status and record it. Input: new status, optional note. Output: updated incident and a history entry.	Only valid transitions (section 4). Each change stores status, who, time and note. Error: 409 invalid transition.	Responder, Admin, System
9. Notifications Create and list in-app notifications; mark as read. Output: notification list and unread count.	A user can read or update only their own notifications. Error: 403.	Any logged-in user
10. Admin dashboard and analytics Totals and charts by category, priority, status, time and location; filters; user and responder management; correct AI classification.	Filters must come from allowed lists; date range valid. Errors: 400 invalid filter, 403 not Admin.	Admin

3. Workflows

3.1 User, Responder and Admin workflows

User	Responder	Admin
1. Register and log in 2. Open dashboard (summary cards, recent reports) 3. Click Report Emergency 4. Enter description, add optional image, choose location (GPS, map or text) 5. Send report and get a report ID 6. View AI recommendation (labelled as a recommendation) 7. Track status on the timeline and read notifications 8. View report history	1. Log in with an account created by Admin 2. Open dashboard (new, assigned, high priority, in progress, resolved) 3. Open an incident: description, image, map, AI analysis 4. Accept the incident 5. Start work and update status 6. Add response notes 7. Mark Resolved 8. Review past incidents	1. Log in 2. View totals and charts; apply filters 3. Watch unresolved and high-priority reports 4. Assign or reassign incidents (especially Pending Assignment) 5. Review AI results and correct category or priority 6. Create and manage responder and user accounts 7. Review incident history

3.2 Incident reporting and AI analysis workflow

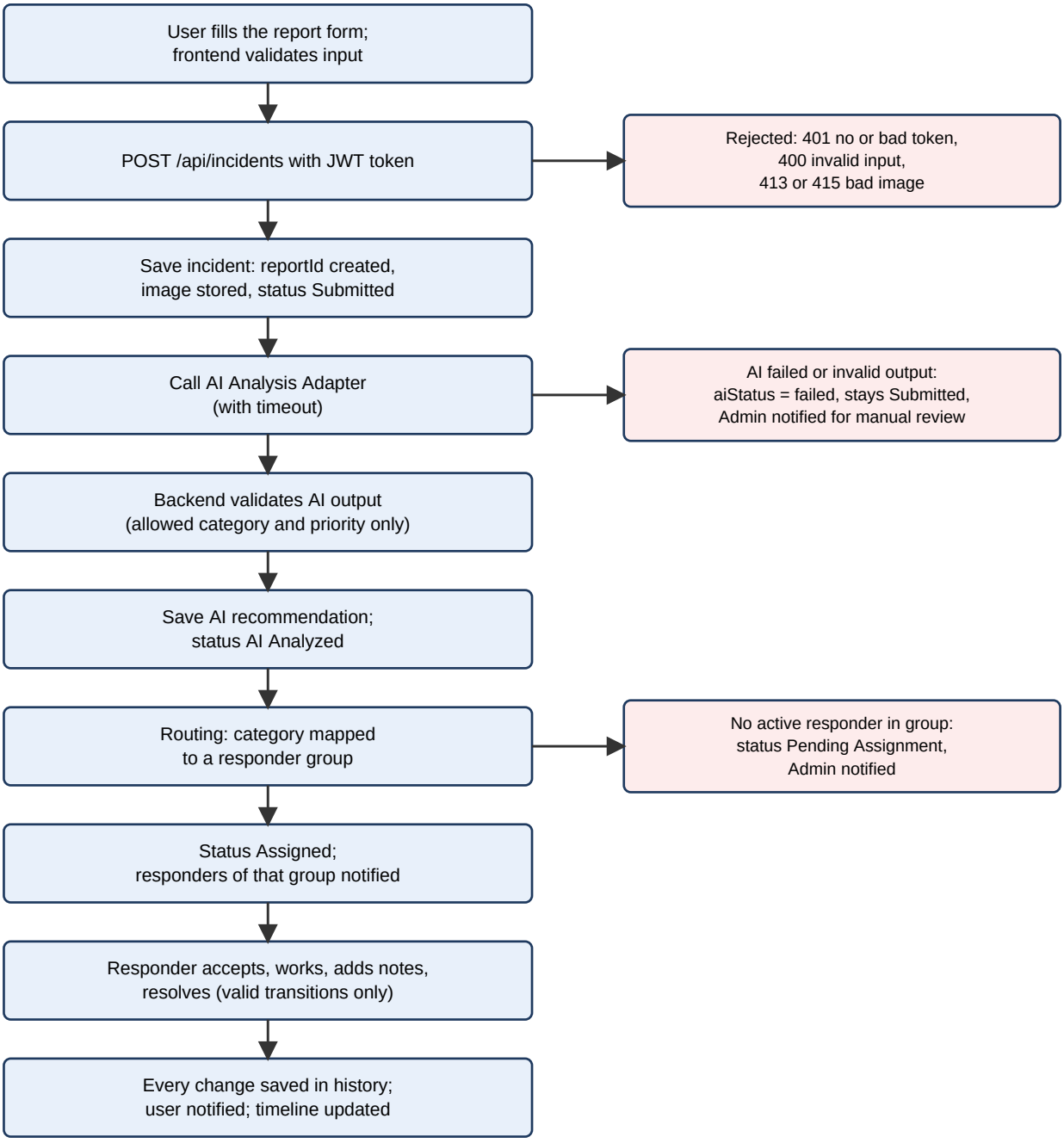
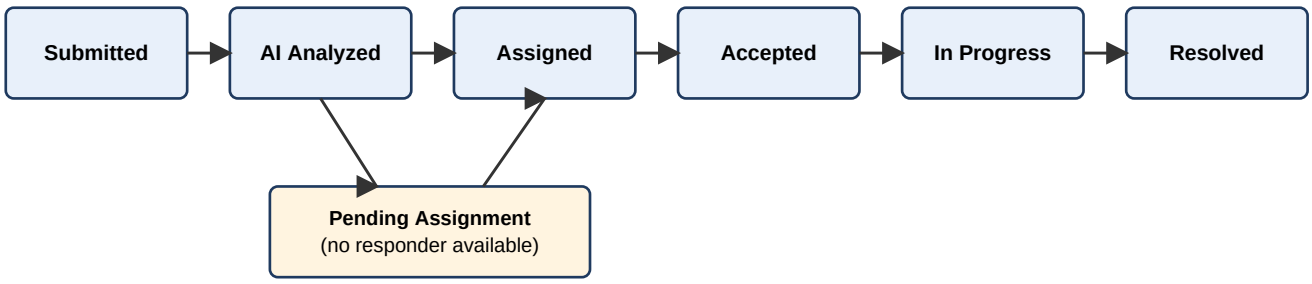


Figure: Incident reporting, AI analysis and routing workflow (proposed). Red boxes show exceptional paths; the report is never lost.

3.3 Responder assignment and status update workflow

- Routing maps the category to a responder group: Fire to Fire; Medical to Medical; Security to Security; Accident to Accident; Infrastructure and Maintenance to Infrastructure; Other to Other (proposed mapping).
- If at least one active responder exists in the group, status becomes Assigned and all of them are notified. The first responder to press Accept becomes the assigned responder (status Accepted); a second attempt gets a 409 message.
- If no active responder exists, status becomes Pending Assignment and the Admin is notified. The Admin can then assign a specific responder.
- The responder moves the incident to In Progress and finally Resolved, adding notes. Each step is checked against the allowed transitions and stored in the history.

4. Status Values and Valid Transitions



From	To	Who or what triggers it
Submitted	AI Analyzed	System, after valid AI output
Submitted	Assigned	Admin, when AI failed (manual review)
AI Analyzed	Assigned	System routing (responder available) or Admin
AI Analyzed	Pending Assignment	System routing (no responder available)
Pending Assignment	Assigned	Admin assigns a responder
Assigned	Accepted	Responder accepts (first one wins)
Accepted	In Progress	Responder starts response
In Progress	Resolved	Responder resolves with a note
Assigned, Accepted, In Progress	Assigned	Admin reassigns to another responder

Any other transition (for example Submitted straight to Resolved) is rejected with 409, even if the client sends it.

5. MongoDB Collection Design

Relationships: Incident.userId refers to User; Incident.assignedResponderId refers to User; IncidentHistory.incidentId refers to Incident and changedBy refers to User; Notification.userId refers to User and incidentId to Incident. References (ObjectId) are used instead of copying data, and private user details are not returned in responder views beyond what is needed.

5.1 User

Field	Type	Notes
`_id`	ObjectId	Primary key
`name`	String	Required
`email`	String	Required, unique, lowercase
`passwordHash`	String	Hash only; never returned by the API
`role`	String	User, Responder or Admin; default User
`responderCategory`	String	Only for Responders: Fire, Medical, Security, Accident, Infrastructure, Other (added to the brief)
`isActive`	Boolean	Inactive responders are skipped in routing (added)
`createdAt, updatedAt`	Date	Timestamps

5.2 Incident

Field	Type	Notes
`_id, reportId`	ObjectId, String	reportId unique and readable, for example INC-2026-0001 (proposed format)
`userId`	ObjectId	Reporter (ref User)
`description`	String	Required, max 2000 characters
`imageUrl`	String	Optional path of the stored image

Field	Type	Notes
aiCategory, aiPriority	String	AI recommendation, kept unchanged
category, priority	String	Current value; starts as the AI value and can be corrected by Admin
aiSummary, aiReason	String	From AI; shown as a recommendation
aiConfidence	Number	Only if the model gives a meaningful value; otherwise empty
aiStatus	String	pending, completed or failed
latitude, longitude	Number	Optional; both present or both empty
locationDescription	String	Typed text such as “Near Block B main entrance”
locationSource	String	gps, map or manual
isUrgent	Boolean	Set when the urgent report button is used
status	String	Submitted, AI Analyzed, Pending Assignment, Assigned, Accepted, In Progress, Resolved
assignedCategory	String	Responder group chosen by routing (added)
assignedResponderId	ObjectId	Set when accepted or assigned by Admin (ref User)
createdAt, updatedAt, resolvedAt	Date	Timestamps; resolvedAt set when Resolved

5.3 IncidentHistory

Field	Type	Notes
_id	ObjectId	Primary key
incidentId	ObjectId	ref Incident
status	String	New status
changedBy	ObjectId	ref User, or empty for system actions
note	String	Optional
timestamp	Date	When the change happened

5.4 Notification

Field	Type	Notes
_id	ObjectId	Primary key
userId	ObjectId	Recipient (ref User)
incidentId	ObjectId	Related incident (ref Incident)
message	String	Text shown in the bell dropdown
type	String	For example submitted, ai_done, assigned, accepted, status_changed, resolved, admin_alert
read	Boolean	Default false
createdAt	Date	Timestamp

5.5 Indexes and data rules

- User.email unique; Incident.reportId unique.
- Incident: (userId, createdAt) for a user’s history; (status, priority) for dashboards and filters; (assignedCategory, status) for the responder list; assignedResponderId.
- IncidentHistory: (incidentId, timestamp). Notification: (userId, read, createdAt).
- Mongoose schema validation enforces required fields, allowed values (enums) and length limits; queries use Mongoose methods with validated input to avoid injection.

6. REST API Endpoints

Proposed routes. “Owner” means the user who created the report. Admin-only and Responder-only routes are protected by role middleware on the server.

Method	Route	Purpose	Access
POST	/api/auth/register	Create a User account	Public
POST	/api/auth/login	Log in and receive a JWT	Public
GET	/api/auth/me	Get the logged-in user’s profile	Any logged-in user
POST	/api/incidents	Submit a report (description, optional image, location); triggers AI analysis and routing	User
GET	/api/incidents	List the logged-in user’s own reports	User
GET	/api/incidents/:id	View one incident with AI result, map data and history	Owner, assigned or same-group Responder, Admin
PUT	/api/incidents/:id	Edit description or location while status is Submitted	Owner, Admin
DELETE	/api/incidents/:id	Remove a mistaken or test report	Admin
GET	/api/responder/incidents	List incidents for the responder’s group and those assigned to them	Responder
PUT	/api/responder/incidents/:id/accept	Accept an incident (first accept wins)	Responder
PUT	/api/responder/incidents/:id/status	Update status (In Progress, Resolved) with an optional note	Responder
POST	/api/responder/incidents/:id/notes	Add a response note	Responder
GET	/api/admin/incidents	List all incidents with filters (date, category, priority, status)	Admin
GET	/api/admin/users	List users and responders	Admin
POST	/api/admin/users	Create a responder or admin account (added)	Admin
PUT	/api/admin/users/:id	Change role, responder category or active flag (added)	Admin
PUT	/api/admin/incidents/:id/assign	Assign or reassign an incident to a responder	Admin
PUT	/api/admin/incidents/:id/classification	Correct AI category or priority (added)	Admin
GET	/api/admin/analytics	Totals and chart data (by category, priority, status, time, location)	Admin
GET	/api/notifications	List the user’s notifications	Any logged-in user
PUT	/api/notifications/:id/read	Mark a notification as read	Any logged-in user

There is no public AI route: the incident controller calls the AI adapter internally, so a user cannot call the AI service directly and the API key is never exposed.

6.1 Notification events

Event	Recipient
Report submitted; AI analysis completed; responder assigned; accepted; status changed; resolved	User (reporter)
New incident for the group; high or critical priority; incident updated	Responder
High-priority incident unassigned; no responder available; AI analysis failed	Admin

7. Authentication and Authorization Flow

- **Register:** frontend validates the form; backend checks the email is unused, hashes the password and saves a User with role User.
- **Login:** backend finds the user by email, compares the password with the stored hash, then signs a JWT containing the user id and role, using a secret kept in an environment variable (proposed expiry: 1 day).
- **Using the token:** the frontend sends it in the Authorization header (Bearer token) on every protected request.
- **Authentication middleware:** verifies the signature and expiry, loads the user and rejects with 401 if invalid.
- **Role middleware:** checks that the role is allowed for the route and rejects with 403 otherwise.
- **Ownership check:** in the controller, a User may open only their own incident; a Responder only incidents of their group or assigned to them.
- **Frontend:** protected React routes redirect by role (User, Responder or Admin dashboard), but the real protection is on the backend.

8. Validation, Error Handling and Security

Error response format (proposed): a JSON object with `success: false`, a short `message` and, for validation errors, a list of field errors. Stack traces and secrets are never returned.

Code	Meaning	Example
400	Invalid input	Empty description, bad coordinates
401	Missing, invalid or expired token; wrong login	Opening the dashboard after the token expired
403	Role or ownership not allowed	User calls an admin route
404	Resource not found	Unknown incident id
409	Conflict or invalid transition	Two responders accept the same incident; Submitted to Resolved
413 / 415	Image too large / wrong type	A PDF uploaded as image
500 / 503	Server error / dependency unavailable	Database down; AI failures are handled without an error to the user

Basic security measures

- Password hashing; JWT secret and AI API key only in backend environment variables.
- Server-side validation of every input; allowed-value checks for category, priority and status.
- Image checks (type, size, safe file name); uploaded files are never executed.
- Output encoding in React and sanitising of text to reduce XSS; Mongoose queries built from validated values.
- CORS limited to the frontend origin; rate limiting on login (proposed).
- Location and image data shown only to the owner, the responsible responders and Admins.